

Riyaaz — Technical Requirements for Engineering Intern

Read this entire document before writing any code. This is your roadmap for the next 8-12 weeks. Every task has a clear scope, a tech decision, and a definition of done. If something is ambiguous, ask — don't assume.

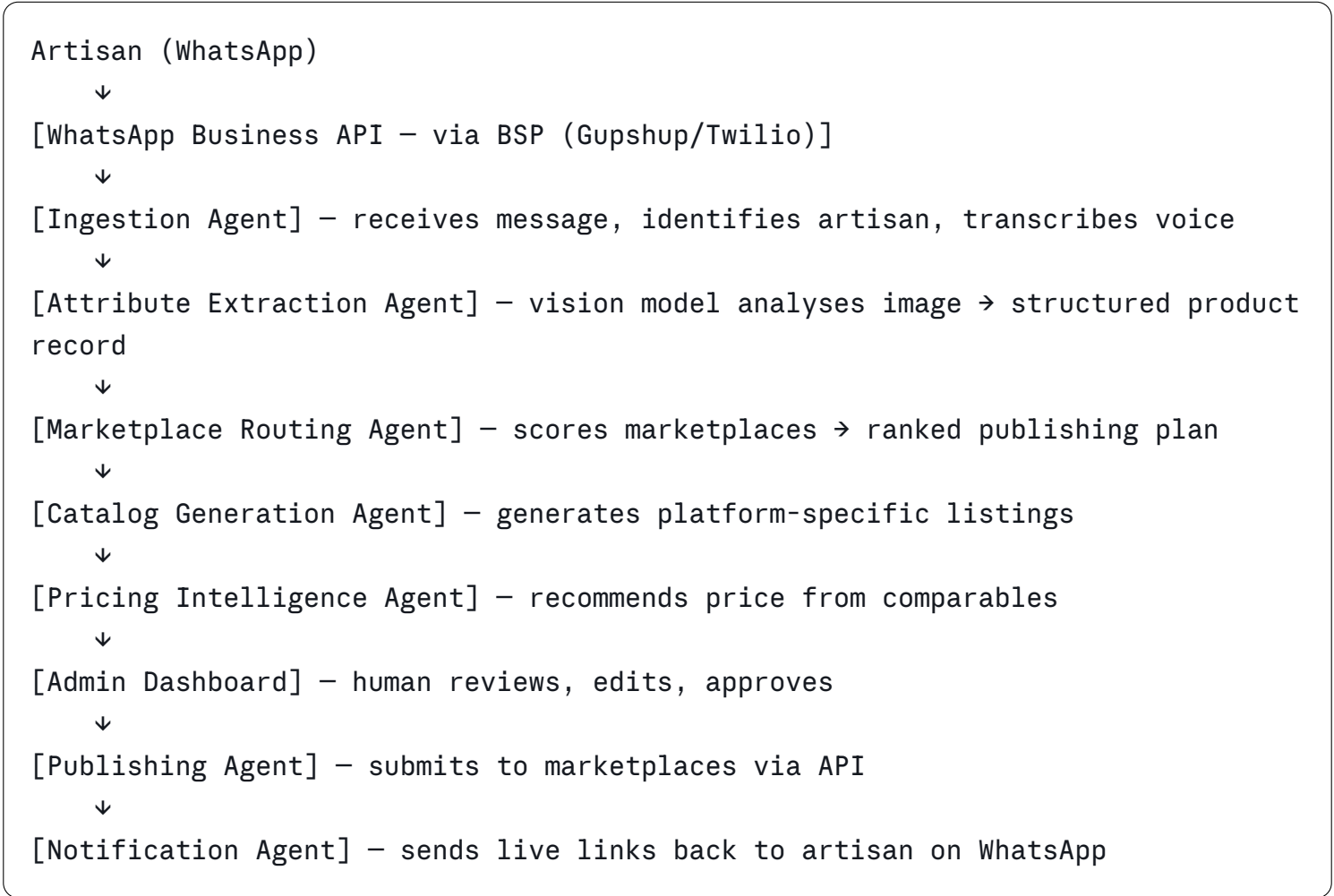
What We're Building

An AI-powered system where an artisan sends a product photo on WhatsApp and gets back live marketplace listings — automatically. Seven AI agents cooperate to extract product attributes, pick marketplaces, generate listings, price the product, publish it, and confirm back to the artisan.

Your job is to build the working MVP of this system.

Read the full PRD (Product Requirements Document) before starting. This document translates that PRD into engineering tasks.

Architecture Overview



Tech Stack (Decided — Do Not Change Without Discussion)

Layer	Technology	Why
Backend	Python (FastAPI)	Best AI/ML library ecosystem. FastAPI is fast, typed, and async-native.
Database	PostgreSQL	Relational. Strong JSON support for flexible product attributes. Battle-tested.
ORM	SQLAlchemy 2.0 + Alembic (migrations)	Type-safe, async support, migration management.
Task Queue	Celery + Redis	Async agent orchestration. Each agent = a Celery task. Retry logic built-in.
AI / LLM	Anthropic Claude API (claude-sonnet-4-20250514)	Vision + text. Used for extraction, catalog generation, pricing rationale.
Voice / ASR	Whisper (OpenAI) or Deepgram	Transcribe artisan voice notes to text. Multilingual.
TTS	Google Cloud TTS or ElevenLabs	Generate voice responses in artisan's language.
WhatsApp	Gupshup or Twilio WhatsApp Business API	BSP for sending/receiving WhatsApp messages. Gupshup preferred (Indian company, better India support).
Admin Dashboard	React + TypeScript	Desktop-first dashboard for admin review.
Storage	AWS S3 (or MinIO for local dev)	Product images, voice notes, generated lifestyle images.
Hosting	AWS (EC2 + RDS + S3) or Railway for MVP	Railway is faster to deploy for MVP. Move to AWS for production.
Containerisation	Docker + Docker Compose	Consistent dev/prod environments.

Project Structure

```
riyaaz/  
├─ docker-compose.yml
```

```

└─ .env.example
└─ README.md

└─ backend/
    └─ main.py                # FastAPI app entry
    └─ config.py              # Environment config
    └─ models/                # SQLAlchemy models
        └─ artisan.py
        └─ product.py
        └─ listing_package.py
        └─ catalog_draft.py
        └─ marketplace.py
        └─ admin.py
        └─ audit_event.py

    └─ agents/                # One file per agent
        └─ ingestion.py
        └─ attribute_extraction.py
        └─ marketplace_routing.py
        └─ catalog_generation.py
        └─ pricing_intelligence.py
        └─ publishing.py
        └─ notification.py

    └─ services/              # Shared services
        └─ whatsapp.py        # WhatsApp API wrapper
        └─ storage.py         # S3/MinIO wrapper
        └─ llm.py             # Claude API wrapper
        └─ speech.py          # ASR + TTS wrapper
        └─ marketplace_connectors/
            └─ base.py         # Abstract connector
            └─ amazon.py
            └─ etsy.py
            └─ ondc.py

    └─ api/                   # FastAPI routers
        └─ webhook.py         # WhatsApp webhook endpoint
        └─ admin.py           # Admin dashboard API
        └─ health.py          # Health check

    └─ tasks/                 # Celery tasks
        └─ orchestrator.py    # Agent pipeline orchestration

    └─ taxonomy/              # Craft taxonomy data
        └─ categories.json    # 200+ Indian craft categories
        └─ gi_tags.json       # GI tag registry

```

```

├── tests/
│   ├── test_agents/
│   ├── test_api/
│   └── fixtures/
├── admin-dashboard/                # React frontend
│   ├── src/
│   │   ├── pages/
│   │   │   ├── Queue.tsx          # Listing review queue
│   │   │   ├── ListingDetail.tsx  # Single listing review
│   │   │   ├── Artisans.tsx       # Artisan directory
│   │   │   └── Analytics.tsx      # Metrics dashboard
│   │   ├── components/
│   │   └── api/
│   └── package.json
├── scripts/
│   ├── seed_taxonomy.py           # Load craft categories
│   ├── seed_marketplaces.py       # Load marketplace configs
│   └── simulate_submission.py      # Test the full pipeline without WhatsApp

```

Sprint Plan (8 Weeks)

Sprint 1 (Week 1-2): Foundation + Ingestion

Goal: A WhatsApp message from a test phone reaches your server, gets acknowledged, and the image + voice note are stored.

Tasks:

1. Set up project scaffolding

- Init git repo with the folder structure above
- Docker Compose with: FastAPI app, PostgreSQL, Redis, MinIO (local S3)
- Create `.env.example` with all required env vars
- Create `README.md` with setup instructions
- Definition of done: `docker-compose up` starts all services, FastAPI health check returns 200

2. Database models (SQLAlchemy)

- Create all models from PRD Section 7: Artisan, Product, ListingPackage, CatalogDraft, Marketplace, Admin, AuditEvent
- Create Alembic migration
- Definition of done: `alembic upgrade head` creates all tables. You can insert and query a test artisan.

3. WhatsApp webhook

- Sign up for Gupshup sandbox (free for testing) or Twilio WhatsApp sandbox
- Create webhook endpoint: `POST /webhook/whatsapp`
- Parse incoming message: extract sender phone, message type (text/image/voice/video), media URLs
- Store raw message in database
- Definition of done: send a WhatsApp message to the sandbox number → see it logged in your database within 5 seconds

4. Ingestion Agent (v1)

- On receiving a WhatsApp image: download the image, upload to S3/MinIO, create a Product record with status "ingested"
- On receiving a voice note: download, store in S3, transcribe using Whisper API, store transcript alongside product
- Send acknowledgement back via WhatsApp: "Received! Processing your product." (in Hindi for now)
- If image is too blurry or too small (<500px), reply: "Photo thoda unclear hai, kya aap ek aur bhej sakte hain?" (Photo is a bit unclear, can you send another?)
- Definition of done: send a photo on WhatsApp → get an acknowledgement → photo is in S3 → Product record exists in DB with status "ingested"

5. Audit logging

- Create utility function `log_event(agent, action, input_data, output_data, product_id)`
- Every agent action must call this. Start with ingestion agent.
- Definition of done: after an ingestion, you can query AuditEvent table and see the full event with timestamps

Sprint 2 (Week 3-4): Extraction + Routing + Pricing

Goal: A product photo gets analysed by AI, attributes extracted, marketplaces ranked, and price recommended.

Tasks:

6. LLM service wrapper

- Create `services/llm.py` with a clean interface:

python

```
async def analyse_image(image_url: str, artisan_context: dict) -> dict:
    # Calls Claude with the image + artisan profile
    # Returns structured product attributes

async def generate_text(prompt: str, system: str) -> str:
    # Generic text generation
```

- Use claude-sonnet-4-20250514 (vision-capable)
- Handle rate limits with exponential backoff
- Definition of done: pass a test craft image → get back a structured JSON with category, materials, technique, colours, dimensions

7. Craft taxonomy

- Create `taxonomy/categories.json` with 200+ Indian craft categories structured as:

json

```
{
  "textiles": {
    "handloom": ["Pochampally Ikat", "Banarasi Silk", "Chanderi", ...],
    "block_print": ["Bagru", "Sanganer", "Bagh", "Ajrakh", ...],
    "embroidery": ["Chikankari", "Kantha", "Kutch", ...]
  },
  "painting": {
    "traditional": ["Madhubani", "Pattachitra", "Warli", "Gond", ...],
    "miniature": ["Kangra", "Basohli", "Rajasthani", ...]
  },
  ...
}
```

- Create `taxonomy/gi_tags.json` with GI tag names, registration years, states, and product descriptions
- These files are referenced by the extraction agent to ground its classification
- Definition of done: both JSON files loaded and queryable. Extraction agent references them in its prompt.

8. Attribute Extraction Agent

- Input: image URL(s) + voice transcript + artisan profile
- Call Claude with a structured prompt:
 - "You are an expert on Indian handicrafts. Analyse this product image..."
 - Include the craft taxonomy as context
 - Include artisan's known craft and location as priors
- Output: structured product record:

python

```
{
  "category": "textiles.handloom",
  "sub_category": "Pochampally Ikat",
  "materials": ["silk", "natural dye"],
  "technique": "double-weft ikat",
  "colour_palette": ["teal", "gold", "cream"],
  "motifs": ["geometric", "diamond"],
  "dimensions": {"length_cm": 550, "width_cm": 120},
  "weight_bucket": "medium",
  "gi_eligible": true,
  "gi_tag": "Pochampally Ikat",
  "quality_grade": "A",
  "provenance_summary": "Handwoven by ...",
  "confidence_scores": {
    "category": 0.95,
    "materials": 0.88,
    "dimensions": 0.60
  }
}
```

- Flag low-confidence fields (< 0.7) for admin attention
- Update Product record with extracted attributes, set status to "extracted"
- Log audit event
- Definition of done: submit a Pochampally ikat photo → get back accurate structured attributes → product record updated in DB

9. Marketplace Routing Agent

- Seed marketplace configs in DB:

python

```
# For each marketplace:
{
    "name": "Amazon Karigar",
    "categories_supported": ["textiles", "painting", "metalwork", ...],
    "price_range": {"min": 500, "max": 50000},
    "buyer_geography": "India + international",
    "commission_rate": 0.15,
    "fulfillment_model": "seller_ships",
    "gi_premium_support": true,
    "return_risk": "medium"
}
```

- Scoring function (deterministic, NOT LLM-based):

python

```
def score_marketplace(product_record, marketplace_config, artisan_profile):
    score = 0
    score += category_fit(product, marketplace) * 0.30
    score += price_band_fit(product, marketplace) * 0.20
    score += buyer_geography_fit(product, marketplace) * 0.15
    score += gi_premium_support(product, marketplace) * 0.15
    score += capacity_fit(artisan, marketplace) * 0.10
    score += return_risk(product, marketplace) * 0.10
    return score
```

- Use LLM only for generating the human-readable rationale per marketplace
- Output: ranked list with recommendation (publish / hold / skip) and rationale
- Definition of done: given a product record → returns ranked marketplace list → rationale is readable and accurate

10. Pricing Intelligence Agent

- For MVP: use Claude to search for comparable products on each marketplace and

For M1, I will use Claude to search for comparable products on each marketplace and suggest pricing

- Prompt structure: "Here is a [product type] from [location]. Compare with similar products on [marketplace]. Consider materials, technique, size, GI status. Recommend price, floor, and ceiling."
- Output:

python

```
{
  "marketplace": "Amazon Karigar",
  "recommended_price": 8500,
  "floor": 6000,
  "ceiling": 12000,
  "rationale": "Comparable Pochampally ikat sarees on Amazon Karigar sell f
  "rationale_artisan_language": "Is size ki Pochampally ikat saree Amazon p
}
```

- Phase 2: replace with actual scraping/API data from marketplaces
- Definition of done: given a product record + marketplace → returns pricing with rationale in English and Hindi

Sprint 3 (Week 5-6): Catalog Generation + Admin Dashboard

Goal: AI generates marketplace-specific listings and an admin can review them.

Tasks:

11. Catalog Generation Agent

- For each marketplace in the publishing plan, generate a complete listing draft:

python

```
{
  "marketplace": "Amazon Karigar",
  "title": "Handwoven Pochampally Ikat Silk Saree - Teal & Gold Geometric P
  "description": "This exquisite Pochampally Ikat saree is handwoven by..."
  "bullet_attributes": [
    "Material: Pure silk with natural dyes",
    "Technique: Double-weft Ikat (UNESCO recognised)",
    ...
  ],
  "keywords": ["pochampally ikat", "handloom saree", "GI tagged", ...],
  "category_path": "Clothing > Women > Ethnic Wear > Sarees > Handloom",
}
```

```

"suggested_price": 8500,
"images": ["original_url", "lifestyle_url_1"],
"provenance_narrative": "Woven by Meena from Pochampally village..."
}

```

- Each marketplace has different schema requirements. Create a prompt template per marketplace:
 - Amazon: title max 200 chars, 5 bullet points, specific browse node
 - Etsy: title max 140 chars, 13 tags, description style is storytelling-heavy
 - ONDC: specific attribute schema per category
- Store templates in `taxonomy/marketplace\_templates/amazon.json`, `etsy.json`, etc.
- Definition of done: given a product record + marketplace routing → generates compliant drafts for 2 marketplaces → drafts pass basic validation (char limits, required fields)

12. Listing Package assembly - Bundle all catalog drafts for one product into a ListingPackage record - Include: product record, routing rationale, pricing per marketplace, catalog drafts, images, provenance - Set status to "pending_review" - Push to admin queue - Definition of done: after full pipeline runs, a ListingPackage exists in DB with all drafts and metadata

13. Admin Dashboard — Queue Page

- React page showing all pending ListingPackages
- Table with columns: artisan name, product thumbnail, craft type, marketplaces recommended, time in queue, status
- Sort by: newest first (default), longest waiting
- Filter by: marketplace, craft category, status
- Click a row → goes to detail page
- Definition of done: page loads in <2 seconds, shows test data, filters work

14. Admin Dashboard — Listing Detail Page

- Shows the full ListingPackage for one product:
 - Product image(s)
 - Extracted attributes (with confidence scores — low confidence highlighted in amber)
 - Marketplace routing decision with rationale
 - For each marketplace: the catalog draft (title, description, bullets, price, keywords)
 - Editable: admin can edit any field in any draft
- Action buttons: "Approve All", "Approve Selected", "Reject" (with reason), "Send

Back to Agent" (triggers re-extraction)

- On approve: set status to "approved", trigger publishing
 - Track edit distance: when admin edits a field, log original value and edited value in AuditEvent
 - Definition of done: admin can view a listing, edit the title, approve it, and the status changes to "approved"
-

Sprint 4 (Week 7-8): Publishing + Notification + End-to-End

Goal: Approved listings get published to at least one real marketplace. Artisan gets the link back on WhatsApp.

Tasks:

15. Marketplace Connector — Base Class

python

```
class MarketplaceConnector:
    async def validate_draft(self, draft: CatalogDraft) -> ValidationResult:
        """Check if draft meets marketplace requirements"""

    async def submit_listing(self, draft: CatalogDraft) -> SubmissionResult:
        """Submit to marketplace API"""

    async def check_status(self, submission_id: str) -> StatusResult:
        """Check if listing is live"""

    async def get_live_url(self, submission_id: str) -> str:
        """Get the public URL of the live listing"""
```

16. Marketplace Connector — Etsy (Start here)

- Etsy has the most accessible API for small sellers
- Register for Etsy API key (developer.etsy.com)
- Implement: create listing, upload images, set attributes, publish
- Handle: draft state, image upload requirements (min 2000px for first image), category taxonomy mapping
- Definition of done: submit a test listing via API → listing appears on Etsy (in draft or active state)

17. Marketplace Connector — Amazon SP-API (if time permits)

- Amazon Selling Partner API is more complex — requires business registration

- For MVP: may need to use CSV upload via Seller Central as a stopgap
- Document the integration path even if full API isn't ready
- Definition of done: either API integration working OR documented CSV upload process with admin instructions

18. Publishing Agent

- On admin approval: pick up the approved ListingPackage
- For each approved marketplace: call the connector's submit_listing
- Handle results:
 - Success: store live URL, set draft status to "live"
 - Transient failure: retry with exponential backoff (max 3 retries)
 - Rejection: set status to "rejected", store reason, push back to admin queue with rejection details
- All actions logged to AuditEvent
- Definition of done: approve a listing in admin dashboard → it appears on Etsy → live URL stored in DB

19. Notification Agent

- On successful publishing: send WhatsApp message to artisan
- Message template (Hindi):



Aapka product live hai!

{product_name} ab {marketplace_name} pe dikh raha hai.
Link: {live_url}

Koi order aayega toh hum aapko yahan batayenge.

- If multiple marketplaces: batch into one message
- Support for voice note response (TTS in artisan's language) – Phase 2
- Definition of done: after publishing, artisan receives WhatsApp with live link within 2 minutes

20. End-to-End Pipeline Test - Create `scripts/simulate_submission.py` that: - Simulates a WhatsApp message with a test craft image - Runs through all 7 agents - Creates a ListingPackage in the admin queue - Admin approves - Publishing agent submits to Etsy - Notification sent - Run this end-to-end and record the time - Target: under 5 minutes total (artisan effort = 0, system time < 3 minutes, admin review < 2 minutes) - Definition of done: one product goes from test image → live on Etsy → confirmation message sent. Screenshot the entire flow.

Sprint 5+ (Week 9-12, if time permits): Enhancements

These are stretch goals. Do them only after Sprint 1-4 are solid.

21. Artisan onboarding flow

- WhatsApp-based onboarding: artisan sends "hi" → bot walks them through registration
- Collect: name, village, craft, phone, language preference, bank details (UPI ID)
- Store in Artisan profile
- Definition of done: new artisan can register entirely via WhatsApp

22. Multilingual support

- Add language detection to Ingestion Agent (detect from voice note or text)
- All artisan-facing messages in detected language (Hindi, Telugu, Odia, Bengali — start with 4)
- Pricing rationale generated in artisan's language
- Definition of done: artisan sends voice note in Telugu → receives confirmation in Telugu

23. Admin analytics page

- Metrics: submissions per day, average time-to-list, edit distance trend, marketplace rejection rate, top crafts
- Simple charts (use Recharts in React)
- Definition of done: analytics page shows real data from test submissions

24. Image enhancement

- Before sending to extraction: auto-crop, auto-white-balance, background removal (use rembg library)
 - Generate lifestyle image using AI image generation from the clean product photo
 - Definition of done: raw phone photo → cleaned product image + lifestyle image generated
-

Environment Setup Instructions

Prerequisites

- Docker and Docker Compose installed
- Python 3.11+
- Node.js 18+
- Accounts: Gupshup or Twilio (WhatsApp), Anthropic (Claude API), AWS or MinIO (storage)

First-Time Setup

```
bash
```

```
# Clone the repo
```

```
git clone <repo-url>
```

```
cd riyaz
```

```
# Copy env file and fill in API keys
```

```
cp .env.example .env
```

```
# Edit .env with your keys
```

```
# Start all services
```

```
docker-compose up -d
```

```
# Run database migrations
```

```
cd backend
```

```
alembic upgrade head
```

```
# Seed taxonomy data
```

```
python scripts/seed_taxonomy.py
```

```
python scripts/seed_marketplaces.py
```

```
# Start the backend
```

```
uvicorn main:app --reload --port 8000
```

```
# In a separate terminal, start Celery worker
```

```
celery -A tasks worker --loglevel=info
```

```
# In a separate terminal, start the admin dashboard
```

```
cd admin-dashboard
```

```
npm install
```

```
npm run dev
```

.env.example

```
# Database
```

```
DATABASE_URL=postgresql://riyaaz:password@localhost:5432/riyaaz
```

```
# Redis
```

```
REDIS_URL=redis://localhost:6379
```

```
# WhatsApp (Gupshup)
```

```
GUPSHUP_API_KEY=
```

```
GUPSHUP_APP_NAME=
```

```
WHATSAPP_WEBHOOK_VERIFY_TOKEN=
```

```
# Anthropic
```

```
ANTHROPIC_API_KEY=
```

```
# Speech (Whisper)
```

```
OPENAI_API_KEY=
```

```
# Storage
```

```
AWS_ACCESS_KEY_ID=
```

```
AWS_SECRET_ACCESS_KEY=
```

```
AWS_S3_BUCKET=riyaaz-media
```

```
AWS_REGION=ap-south-1
```

```
# For local dev with MinIO:
```

```
S3_ENDPOINT_URL=http://localhost:9000
```

```
# Etsy
```

```
ETSY_API_KEY=  
ETSY_API_SECRET=  
ETSY_SHOP_ID=
```

```
# Amazon SP-API (Phase 2)  
AMAZON_SP_CLIENT_ID=  
AMAZON_SP_CLIENT_SECRET=  
AMAZON_SP_REFRESH_TOKEN=
```

Testing Requirements

- Every agent must have at least 3 test cases:
 - Happy path (good image, good data)
 - Edge case (blurry image, missing voice note, unusual craft)
 - Error case (API timeout, invalid data)
 - Use pytest. Fixtures in `tests/fixtures/` (include 5-10 sample craft images)
 - Admin dashboard: manual testing is fine for MVP. Write down test scenarios.
 - End-to-end test: `simulate_submission.py` must pass before any PR is merged to main.
-

Code Quality Rules

1. **Type hints everywhere.** Every function has typed parameters and return type.
 2. **Docstrings on every public function.** One line explaining what it does.
 3. **No hardcoded strings.** All prompts in separate files. All config in `.env`.
 4. **Every agent logs an AuditEvent.** No exceptions. This is how we debug in production.
 5. **Every external API call has a timeout and retry.** Default timeout: 30 seconds. Max retries: 3.
 6. **PR descriptions must include:** what changed, why, how to test, and a screenshot if UI changed.
-

Questions You Should Ask Before Starting

1. Do we have access to a Gupshup/Twilio sandbox account? (If not, you can build the webhook and test with Postman/curl simulating WhatsApp payloads while waiting for